



OBJETIVOS

- Diseñar e implementar un sistema embebido interactivo que integre entradas digitales, salidas luminosas, visualización numérica y restricciones temporales, y que tenga un comportamiento verificable.
- Desarrollar dos implementaciones funcionalmente equivalentes del sistema, una mediante Arduino y otra mediante MicroPython, conservando el hardware, los requisitos y las pruebas de aceptación.
- Estructurar y documentar el software con responsabilidades e interfaces claras, de modo que pueda comprenderse, verificarse y mantenerse en ambos entornos.
- Comparar experimentalmente los procesos de desarrollo en Arduino y MicroPython y formular conclusiones sustentadas sobre la complejidad, ventajas, desventajas y condiciones de uso de cada alternativa.

DESCRIPCIÓN

Se desarrollará un juego de memoria en el que el jugador observa una secuencia luminosa y debe reproducirla mediante pulsadores. La longitud de la secuencia aumenta con el nivel, mientras que su velocidad de presentación aumenta de manera progresiva. El sistema administra vidas, limita el tiempo de respuesta y conserva el tiempo acumulado del jugador.

Interfaz del sistema

El sistema estará compuesto por cuatro LEDs para presentar la secuencia, cuatro pulsadores asociados individualmente a estos LEDs, un quinto LED para representar el tiempo disponible y un pulsador independiente para iniciar y reiniciar la partida. La información se mostrará en cuatro dígitos de 7 segmentos: uno para el nivel, uno para las vidas y dos para el tiempo acumulado de respuesta.

Elemento	Función observable
LEDs 1 a 4	Presentar la secuencia y confirmar visualmente cada entrada reconocida.
Pulsadores 1 a 4	Permitir al jugador reproducir la secuencia.
LED 5	Indicar, mediante un pálpito de velocidad creciente, el avance del tiempo disponible.
Pulsador de inicio/reinicio	Iniciar una partida y solicitar el reinicio mediante una pulsación sostenida.
Display de nivel	Mostrar el nivel actual, de 1 a 9.
Display de vidas	Mostrar las vidas restantes, con un valor inicial de 3.
Dos displays de tiempo	Mostrar el tiempo acumulado de respuesta, entre 00 s y 99 s.

Inicio y reinicio

Después de energizar o reiniciar el sistema, el juego permanecerá en espera y mostrará nivel 1, 3 vidas y tiempo acumulado de 00. Al pulsar el botón de inicio comenzará la partida. Durante una partida o después de su finalización, una pulsación sostenida de dos segundos cancelará el estado actual y restablecerá los valores iniciales; será necesaria una nueva pulsación para comenzar.

Secuencia y niveles

Cada elemento de la secuencia corresponderá a uno de los cuatro LEDs. En el nivel 1, la secuencia tendrá un único elemento. Al completar un nivel, la secuencia anterior se conservará y se añadirá un nuevo elemento al final. En el nivel n, la secuencia tendrá n elementos y deberá ser impredecible para el jugador. Esto implica que después de reiniciar el juego o el sistema completo, la secuencia debería cambiar.

Durante la presentación, los pulsadores asociados a la secuencia no generarán entradas de juego (están desactivados). Los elementos consecutivos deberán distinguirse claramente, incluso cuando correspondan al mismo LED.

La frecuencia inicial de presentación será de 1 Hz y aumentará en 0,5 Hz cada dos niveles:

Nivel	Frecuencia de presentación
1-2	1,0 Hz
3-4	1,5 Hz
5-6	2,0 Hz
7-8	2,5 Hz
9	3,0 Hz

Para un nivel n y una frecuencia f(n), el tiempo de presentación será $T_{\text{presentación}} = n / f(n)$.

Ingreso y tiempo de respuesta

Una vez concluida la presentación, el sistema habilitará el ingreso de la respuesta. Cada pulsación válida se reconocerá como una única entrada y se encenderá brevemente el LED asociado, independientemente del tiempo durante el cual el pulsador permanezca presionado.

El jugador dispondrá de un tiempo máximo equivalente al 125 % del tiempo de presentación: $T_{\text{máximo}} = 1,25 \cdot T_{\text{presentación}}$. Durante este intervalo, el quinto LED emitirá un pálpito cuya velocidad aumentará de forma perceptible hasta el vencimiento. El indicador se apagará cuando la secuencia se complete, se detecte una entrada incorrecta o se agote el tiempo.

Resultado del intento y final de partida

El intento será correcto cuando el jugador reproduzca todos los elementos, en el orden establecido y dentro del tiempo permitido. En ese caso, el nivel aumentará, se añadirá un nuevo elemento a la secuencia y comenzará el siguiente nivel.

El intento será incorrecto si el jugador selecciona un elemento distinto del esperado o si transcurre el tiempo máximo. Se descontará una vida y, si aún quedan vidas, se repetirá el mismo nivel con la misma secuencia.

Cada equipo deberá definir y documentar señales claramente diferenciables para: secuencia correcta, entrada incorrecta, agotamiento del tiempo, pérdida de una vida, finalización exitosa del nivel 9 y finalización por agotamiento de vidas. Estas señales no deberán alterar las reglas del juego, los tiempos ni el estado de la partida.

La partida terminará al completar correctamente el nivel 9 o al perder las tres vidas. Al finalizar, los displays conservarán el nivel alcanzado, las vidas restantes y el tiempo acumulado hasta que se solicite un reinicio.

Tiempo acumulado

Los dos dígitos destinados al tiempo mostrarán la suma de los intervalos utilizados para responder en todos los intentos. El conteo comenzará al terminar cada presentación y finalizará al completar la secuencia, al ingresar un elemento incorrecto o al agotar el tiempo. Se incluirán los intentos correctos, incorrectos y vencidos; se excluirá el tiempo de presentación. El valor se mostrará entre 00 s y 99 s y permanecerá en 99 si se supera ese límite.

Implementaciones requeridas

El sistema deberá implementarse en dos entornos: una versión en Arduino y otra en MicroPython. Ambas deberán conservar la misma interfaz física, el mismo comportamiento observable y los mismos criterios de aceptación. No se aceptará que una versión reduzca funciones, niveles, señales o restricciones temporales para facilitar su desarrollo.

PROCEDIMIENTO

El desarrollo deberá seguir la Guía metodológica para el desarrollo de prácticas del curso de Electrónica Digital III. Este procedimiento identifica los hitos particulares de la práctica, pero no reemplaza las fases, definiciones ni criterios de trazabilidad establecidos en dicha guía.

Secuencia sugerida de trabajo

1. Interpretar la descripción, delimitar el alcance y declarar los supuestos necesarios. Resolver con el docente cualquier ambigüedad que modifique el comportamiento esperado.
2. Derivar los requisitos funcionales y no funcionales. Definir las pruebas de aceptación antes de iniciar la implementación y establecer una matriz de trazabilidad.
3. Identificar las decisiones de diseño relevantes, plantear alternativas, definir criterios de comparación y documentar las selecciones. El diseño deberá considerar desde el comienzo ambas implementaciones requeridas.
4. Elaborar el diagrama de bloques, el esquema eléctrico, la asignación de recursos y la estructura del software. Las responsabilidades e interfaces entre componentes deberán ser explícitas.
5. Verificar por separado los elementos de entrada, de salida, de visualización y de temporización. Registrar resultados e incidencias antes de integrar el juego completo.
6. Construir e integrar la versión Arduino. Ejecutar pruebas de componente y corregir las desviaciones encontradas sin modificar retrospectivamente los criterios de aceptación.
7. Construir e integrar la versión de MicroPython en el mismo hardware y con la misma especificación. Las diferencias internas son admisibles; las diferencias funcionales no lo son.
8. Ejecutar en ambas versiones el mismo plan de pruebas de aceptación, bajo condiciones comparables, y conservar evidencia suficiente para establecer el cumplimiento de cada requisito.
9. Comparar los entornos de desarrollo a partir de la experiencia y de la evidencia obtenida. Formular conclusiones que distingan las observaciones del equipo, los resultados medidos y las propiedades generales de cada ecosistema.
10. Revisar la trazabilidad, completar el informe, generar la documentación del código y preparar la entrega con la estructura solicitada.

Estructura y documentación del software

El software deberá estar organizado en componentes con responsabilidades definidas y dependencias controladas. La estructura debe separar, como mínimo, el acceso a los dispositivos, la lógica del juego y la coordinación general de la aplicación, aunque el equipo podrá proponer otra organización si la justifica y mantiene una separación equivalente.

El código de ambas implementaciones deberá estar documentado en un formato procesable por Doxygen. La entrega incluirá la configuración utilizada y la documentación generada. La documentación deberá describir módulos, responsabilidades, interfaces, parámetros, valores de retorno, efectos secundarios y supuestos relevantes; no basta con copiar literalmente el código.

Informe técnico

Se entregará un único informe que documente ambas implementaciones y permita reconstruir el proceso de desarrollo. Debe contener, como mínimo:

- Alcance, supuestos, restricciones y descripción operativa del sistema.
- Requisitos funcionales y no funcionales con identificadores únicos.
- Pruebas de aceptación y matriz de trazabilidad requisito-decisión-prueba-evidencia.
- Decisiones de diseño, alternativas consideradas, criterios, evidencia, selección y consecuencias.
- Diagrama de bloques, esquema eléctrico y asignación de recursos del hardware.
- Arquitectura del software e interfaces entre componentes para ambas implementaciones.
- Resultados de las pruebas de componente, de integración y de aceptación, incluidas las desviaciones encontradas.
- Evidencias comparables de funcionamiento en Arduino y MicroPython.
- Comparación crítica de los entornos de desarrollo y conclusiones.
- Referencias y anexos necesarios para reproducir las pruebas.

3.4 Comparación Arduino-MicroPython

La comparación no deberá limitarse a preferencias personales ni a indicar cuál código tiene menos líneas. Deberá contrastar, al menos, los siguientes aspectos con ejemplos o evidencia del trabajo realizado:

Dimensión	Preguntas orientadoras
Preparación del entorno	¿Qué herramientas, controladores, configuraciones y pasos fueron necesarios para iniciar?
Ciclo de desarrollo	¿Cómo se editó, cargó, ejecutó y corrigió cada versión? ¿Qué tan inmediata fue la retroalimentación?
Expresividad y complejidad	¿Cómo afectaron el lenguaje, las bibliotecas y el modelo de ejecución a la solución?
Diagnóstico de errores	¿Qué información ofreció cada entorno ante errores de sintaxis, de ejecución, de temporización o de interacción con el hardware?
Control del hardware	¿Qué nivel de acceso, previsibilidad y dependencia de las abstracciones se observó?

Dimensión	Preguntas orientadoras
Comportamiento temporal	¿Qué diferencias medibles aparecieron en tiempos, estabilidad o respuesta?
Uso de recursos	¿Qué diferencias relevantes se observaron en memoria, almacenamiento, tiempo de arranque u otros recursos disponibles?
Estructura y mantenimiento	¿Qué tan fácil fue modularizar, documentar, probar y modificar cada solución?
Contexto de uso	¿Para qué tipos de problemas resultaría preferible cada entorno y bajo qué restricciones?

Cuidado interpretativo: Una diferencia observada en una implementación no demuestra por sí sola una propiedad universal de Arduino o MicroPython. Las conclusiones deberán considerar la experiencia del equipo, las decisiones tomadas, las bibliotecas utilizadas y las limitaciones del experimento.

Entrega

La entrega se realizará en Moodle antes de la medianoche del lunes 7 de septiembre, mediante un único archivo comprimido. El archivo deberá contener el informe en formato PDF, el código fuente completo de ambas implementaciones, los archivos de configuración necesarios, la documentación generada con Doxygen y las evidencias complementarias que no estén incorporadas en el informe.

La estructura de carpetas deberá permitir identificar con claridad las versiones de Arduino y MicroPython. No se aceptarán únicamente archivos compilados, capturas de pantalla del código ni enlaces que requieran permisos adicionales.

EVALUACIÓN

La calificación considerará el producto funcional y la calidad del proceso de ingeniería. El cumplimiento parcial de una versión no podrá compensarse mediante una comparación puramente narrativa.

Criterio	Peso	Desempeño completo	Desempeño parcial	Insuficiente
Implementación Arduino	15 %	Cumple todos los requisitos y supera las pruebas previstas.	Presenta fallas menores o evidencia incompleta sin comprometer el núcleo del juego.	Faltan funciones esenciales, no ejecuta o no puede verificarse.
Implementación MicroPython	15 %	Cumple todos los requisitos y supera las pruebas previstas.	Presenta fallas menores o evidencia incompleta sin comprometer el núcleo del juego.	Faltan funciones esenciales, no ejecuta o no puede verificarse.

Criterio	Peso	Desempeño completo	Desempeño parcial	Insuficiente
Equivalencia funcional y temporal	10 %	Mismo comportamiento, hardware y plan de pruebas; diferencias medidas y explicadas.	Existen diferencias menores o comparación temporal limitada.	Las versiones no son comparables o se evaluaron bajo condiciones distintas.
Aplicación de la metodología	15 %	Existe trazabilidad coherente; requisitos, decisiones y pruebas preceden y sustentan la implementación.	La metodología es reconocible, pero los vínculos son débiles o los productos resultan incompletos.	El proceso se reconstruyó después de programarlo o carece de trazabilidad.
Pruebas y evidencia	10 %	Pruebas reproducibles, criterios objetivos y evidencia suficiente para ambas versiones.	Pruebas pertinentes con vacíos en las condiciones, la medición o la evidencia.	Se afirma que funciona, pero no hay pruebas reproducibles ni evidencia suficiente.
Arquitectura y calidad del código	10 %	Responsabilidades e interfaces claras, baja duplicación y estructura coherente en ambos proyectos.	Estructura funcional con acoplamiento o con inconsistencias moderadas.	Código monolítico, difícil de seguir o con dependencias no controladas.
Documentación Doxygen	5 %	Cobertura útil y consistente; configuración y salida generadas incluidas en ambas versiones.	Documentación generada con omisiones o comentarios poco informativos.	No se genera, no cubre ambas versiones o se limita a repetir el código.
Informe y análisis comparativo	20 %	Informe completo, preciso y crítico; comparación sustentada, limitaciones explícitas y conclusiones defendibles.	Informe comprensible, con análisis principalmente descriptivo o con evidencias incompletas.	Informe incompleto, comparación basada en preferencias o conclusiones sin sustento.

La ausencia total de una de las dos implementaciones impide valorar la equivalencia funcional y limita el criterio de análisis comparativo, aunque el informe describa el entorno que falta.